

Lab08 - Using RSA for Keys & Certificates

This week we will be working with public/private keys and X.509 certificates. We will learn how to generate RSA keys for use in SSH, view their specific parameters, and use them to encrypt, decrypt, and verify messages. We will also learn how to generate certificates, send them off to be signed by a certificate authority, and how to use them in signing and encrypting email messages. Instructions for submission are listed at the end of the document. All exercises will be performed on your **Ubuntu desktop IN ISELab**. **Note:** you'll need to perform all of the steps in order.

*all instances of <netid> should be replaced by **YOUR** netid:

vi <netid>_msg.txt ⇒ vi jrursch_msg.txt

Part 1

1. Before you start this lab, be sure to run the `noproxy.sh` script.
 - a. `./noproxy.sh <the first three octets of your network>`
2. First, you will need to create a working directory to hold the documents generated during this homework. Create a new directory and change-directory into it:
 - a. `mkdir ~/homework/lab08`
 - b. `cd lab08`
3. Next, generate a public/private key pair on your machine using `openssl`. You could also use `python3` to generate the key pair, but `openssl` is a commonly used program and this lab will give you some experience using it. To generate the private key, execute the following command:
 - a. `openssl genpkey -algorithm RSA -out <netid>_private_key.pem -pkeyopt rsa_keygen_bits:2048`
 - b. This will generate a single file containing your private key in PEM format. You can `cat` the file to read the private key in base64.
3. Extract your public key from the private key:
 - a. `openssl rsa -pubout -in <netid>_private_key.pem -out <netid>_public_key.pem`
4. View all of your private key's data with:
 - a. `openssl rsa -text -in <netid>_private_key.pem |less`
 - b. This will show the modulus, exponents (public and private), primes (p and q) that were used to compute your key pair. It also shows exponents 1 and 2, as well as the coefficient, that are used in the Chinese Remainder Theorem which we briefly

- covered in class. CRT is used during decryption with RSA because it is faster than exponentiation. **Submit a screenshot** of the output of this command.
- c. Generate one more key pair (repeat steps 2-4). **Comment on** what the various values mean in the textual output. 1) Which ones are constant throughout? 2) Which ones vary? 3) What do these values represent?
5. Use `ftp` to copy your public key to `certs.homework.3310.com`. This will allow auto-magically generated messages to be signed with your public key.
- a. From within the directory you created your public/private key pairs, enter the command: `ftp`
 - b. Type: `open certs.homework.3310.com` and hit enter
 - c. Enter username and password: `anonymous` and hit enter
 - d. Change directory to `upload_rsa_keys` with the following command:
 - i. `cd upload_rsa_keys`
 - e. Type: `put <netid>_public_key.pem` to upload your public key.
 - f. **Please note:** The key you upload should be in your current working directory and in the following format (should be if you've been following the instructions so far):
 - i. `<netid>_public_key.pem`
 - g. Close connection with `close` and `quit`
 - h. If you need to re-upload a file, you can delete your public key (while logged in as anonymous) with the following command:
 - i. `delete <netid>_public_key.pem`
6. The server should automatically add your public key to the `authorized_keys` file for the `ssh_guest` user. We can now use SSH and its supported tools (SFTP, `rsync`) to retrieve/put files on our server.
- a. Change the permissions on your private key: `chmod 600 <netid>_private_key.pem`
 - b. Type the following command to open a sftp session. We will be connecting via a shared account to retrieve our messages.
 - i. `sftp -i <netid>_private_key.pem ssh_guest@certs.homework.3310.com`
 - ii. You may be faced with a message about the authenticity of the host - just follow the on-screen instructions. Should only happen the first time connecting.
 - c. Once connected, `cd` into your "messages" directory
 - i. `cd lab08/rsa_msgs`
 - d. Copy your "messages" directory to your machine:
 - i. `get -r <netid>`
 - e. Exit the session via: `exit`
 - f. This should recursively copy a directory containing five automatically generated messages, the Certificate Authority's public certificate and "lab08's" public certificate to your virtual desktop. There is also a signature file "sig.txt.sha256" that will allow you to test the authenticity of each of the messages.

- g. Copy the cacert.pem and the lab08_public_key.pem files to your lab08 directory on your 3310 VM to make finding them easier for future steps (keep the “..” at the end of the commands).
 - i. `cd <netid>`
 - ii. `cp lab08_public_key.pem ..`
 - iii. `cp cacert.pem ..`
- h. Google the differences between FTP and SFTP. **Answer the following questions:** 1) Why would you want one over the other? 2) Why did we need to specify our private key? 3) What protection does public/private key authentication offer?
7. Use the following command to verify each of your five messages that are inside your <netid> folder.
 - a. `openssl dgst -sha256 -verify lab08_public_key.pem -signature sig.txt.sha256 <filename>.txt 2>/dev/null`
 - b. This command compares the hash of the specified message against the provided signature. Four messages have been “modified” and one is the true message. Determine which message has not been modified and include **a screenshot of all 5 success/failures** in your homework submission. Your answer may vary from others in the class. ****By default there is debugging information provided by openssl dgst command, therefore the stderr is being redirected to /dev/null. It makes the output cleaner to read.*
 - c. **Comment on** what you’ve just done by explaining what the openssl command is doing and why this is significant.
 - 1) What is known about the message?
 - 2) What is the message protected against and what is it vulnerable to?
8. Now you will generate your own message and encrypt it.
 - a. Change directory back to your lab08 folder: `cd ~/homework/lab08`
 - b. Using your favorite text editor (nano, vim, Text Editor, etc...) create a plain-text message and save it to your lab08 directory as `plain.txt`
 - c. Encrypt the message using the provided public key as follows:
 - i. `openssl pkeyutl -encrypt -inkey lab08_public_key.pem -pubin -in plain.txt -out <netid>_encrypt.dat`
 - d. Using SFTP, upload this file to the certs server
 - i. `sftp -i <netid>_private_key.pem ssh_guest@certs.homework.3310.com`
 - ii. `cd lab08/encrypted_msgs`
 - iii. `put <netid>_encrypt.dat`
 - iv. `exit`
 - e. **Comment on** what this message is protected against and what it is vulnerable to (compared to the message we downloaded in step 6).
9. Generate a X.509 certificate signing request to be submitted using your RSA private key
 - a. Ensure that you are in your lab08 folder: `cd ~/homework/lab08`

- b. `openssl req -new -key <netid>_private_key.pem -out <netid>_certificate.csr -days 3650`
 - c. You should be prompted to answer a series of questions. Ensure that you answer these as follows:
 - i. Country Name: US
 - ii. State or Province Name: Iowa
 - iii. Locality Name: Ames
 - iv. Organization Name: 3310.com
 - v. Organizational Unit Name: homework
 - vi. Common Name: <netid>.homework.3310.com
 - vii. Email Address: <netid>@homework.3310.com
 - d. Ensure that you **do not** enter a challenge password or optional company name. Just press “Enter” when presented with these fields. If you mess up, you can always cancel with “Ctrl+C”
10. Upload your CSR to be signed, similar to uploading your public key earlier
- a. Start the interactive FTP program: `ftp`
 - b. Open a connection: `open certs.homework.3310.com`
 - c. Log in as: `anonymous`
 - d. Change directory to: `upload_cert_req`
 - e. Upload your CSR: `put <netid>_certificate.csr`
 - f. Note: You won’t be able to do this twice. If you make a mistake and have to do it over, reach out to the TA. They must remove you from the certificate database (index.txt), as well as remove your csr and pem from the corresponding ftp directories.
11. Retrieve your signed certificate
- a. While still logged into FTP, change directory to: `download_cert` via the command:
 - i. `cd ../download_cert`
 - b. Retrieve your certificate with: `get <netid>_certificate.pem`
 - i. It may take a minute or so to generate your certificate in the directory.
PLEASE BE PATIENT.
 - c. Close connection: `close; quit;`
12. View your certificate
- a. Run the following command to view the contents of your newly signed certificate:
 - b. `openssl x509 -in <netid>_certificate.pem -text -noout | less`
 - c. **Include** screenshots of the output
 - d. Do any parts of the certificate match with your private key? If so, why?
 - e. What was happening during the Certificate Signing process? Why did you need to submit it for signing?

Lab 08 Template

Some questions require multiple parts to be answered, be sure to discuss them in full.

Part 01:

1. **Screenshot of the output from** `openssl rsa -text -in <netid>_private_key.pem`
(10 points)
2. **Comparison of the same/different values observed across the extra generated keys**
 - i. Which values are constant?
 - ii. Which ones vary?
 - iii. What do these values represent?(15 points, 5 points each question)
3. **Discussion of the differences between FTP and SFTP.**
 - i. Why would you want one over the other?
 - ii. Why did we need to specify our private key?
 - iii. What protection does this offer?(15 points, 5 points each question)
4. **Screenshot of the five messages [netid]1.txt, [netid]2.txt, ... [netid]5.txt**
(10 points)
5. **Discussion on hash verification**
 - i. What is known about the message?
 - ii. What is the message protected against and what is it vulnerable to?(10 points, 5 points each question)
6. **Discussion on what the message generated in step 8e protected against and what it is vulnerable to (compared to the message we downloaded in step 6).**
(10 points, 5 points each part)
7. **Screenshot of the signed certificate ([netid]_certificate.pem) when looked at through openssl**
(10 points)
8. **Discussion from step 12**

- i. **Do any parts of the certificate match with your private key? If so, why?**
- ii. **What was happening during the Certificate Signing process? Why did you need to submit it for signing?**

(20 points, 10 points each answer, 10 points each why)